



```
In [3]: # pip install pandas
```

```
In [4]: import pandas as pd
```

```
In [8]: a=[12,34,6]
        b=["a","b","c"]
```

```
In [9]: a
```

```
Out[9]: [12, 34, 6]
```

```
In [38]: s=pd.Series(a, index=b)
```

```
In [11]: pd.Series(b)
```

```
Out[11]: 0    a
         1    b
         2    c
         dtype: object
```

```
In [13]: data={
          "name":["rahul","kamal","raman"],
          "age": [23,45,67]
        }
```

```
In [14]: data
```

```
Out[14]: {'name': ['rahul', 'kamal', 'raman'], 'age': [23, 45, 67]}
```

```
In [15]: pd.DataFrame(data)
```

```
Out[15]:
```

|   | name  | age |
|---|-------|-----|
| 0 | rahul | 23  |
| 1 | kamal | 45  |
| 2 | raman | 67  |

```
In [28]: data2 = {
          'Full_Name': ['John', 'Anna', 'Peter', 'Linda'],
          'Age': [28, 34, 29, 42],
          'City': ['New York', 'Paris', 'Berlin', 'London'],
          'Salary': [65000, 70000, 62000, 85000]
        }
```

```
In [29]: df= pd.DataFrame(data2)
```

```
In [30]: df
```

```
Out[30]:
```

|   | Full_Name | Age | City     | Salary |
|---|-----------|-----|----------|--------|
| 0 | John      | 28  | New York | 65000  |
| 1 | Anna      | 34  | Paris    | 70000  |
| 2 | Peter     | 29  | Berlin   | 62000  |
| 3 | Linda     | 42  | London   | 85000  |

```
In [32]: df.Full_Name
```

```
Out[32]: 0    John
1    Anna
2    Peter
3    Linda
Name: Full_Name, dtype: object
```

```
In [34]: df[["Full_Name"]]
```

```
Out[34]:
```

|   | Full_Name |
|---|-----------|
| 0 | John      |
| 1 | Anna      |
| 2 | Peter     |
| 3 | Linda     |

```
In [35]: df["Full_Name"]
```

```
Out[35]: 0    John
1    Anna
2    Peter
3    Linda
Name: Full_Name, dtype: object
```

```
In [36]: df[["Full_Name", "Age"]]
```

```
Out[36]:
```

|   | Full_Name | Age |
|---|-----------|-----|
| 0 | John      | 28  |
| 1 | Anna      | 34  |
| 2 | Peter     | 29  |
| 3 | Linda     | 42  |

```
In [37]: df.loc[0]
```

```
Out[37]: Full_Name      John
        Age           28
        City          New York
        Salary        65000
        Name: 0, dtype: object
```

```
In [39]: s
```

```
Out[39]: a    12
        b    34
        c     6
        dtype: int64
```

```
In [41]: s.loc["a"]
```

```
Out[41]: np.int64(12)
```

```
In [42]: s.iloc[0]
```

```
Out[42]: np.int64(12)
```

```
In [43]: df
```

```
Out[43]:
```

|   | Full_Name | Age | City     | Salary |
|---|-----------|-----|----------|--------|
| 0 | John      | 28  | New York | 65000  |
| 1 | Anna      | 34  | Paris    | 70000  |
| 2 | Peter     | 29  | Berlin   | 62000  |
| 3 | Linda     | 42  | London   | 85000  |

```
In [44]: df["color"]=["red", "green", "yellow","black"]
```

```
In [45]: df
```

```
Out[45]:
```

|   | Full_Name | Age | City     | Salary | color  |
|---|-----------|-----|----------|--------|--------|
| 0 | John      | 28  | New York | 65000  | red    |
| 1 | Anna      | 34  | Paris    | 70000  | green  |
| 2 | Peter     | 29  | Berlin   | 62000  | yellow |
| 3 | Linda     | 42  | London   | 85000  | black  |

```
In [48]: df.drop("color", axis="columns", inplace=True)
```

```
In [49]: df
```

```
Out[49]:
```

|   | Full_Name | Age | City     | Salary |
|---|-----------|-----|----------|--------|
| 0 | John      | 28  | New York | 65000  |
| 1 | Anna      | 34  | Paris    | 70000  |
| 2 | Peter     | 29  | Berlin   | 62000  |
| 3 | Linda     | 42  | London   | 85000  |

```
In [51]: df["Full_Name"][0]="python"
```

C:\Users\rohan\AppData\Local\Temp\ipykernel\_79560\4092999776.py:1: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!  
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row\_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user\\_guide/indexing.html#returning-a-view-versus-a-copy](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
df["Full_Name"][0]="python"
```

C:\Users\rohan\AppData\Local\Temp\ipykernel\_79560\4092999776.py:1: SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user\\_guide/indexing.html#returning-a-view-versus-a-copy](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
df["Full_Name"][0]="python"
```

```
In [52]: df
```

```
Out[52]:
```

|   | Full_Name | Age | City     | Salary |
|---|-----------|-----|----------|--------|
| 0 | python    | 28  | New York | 65000  |
| 1 | Anna      | 34  | Paris    | 70000  |
| 2 | Peter     | 29  | Berlin   | 62000  |
| 3 | Linda     | 42  | London   | 85000  |

```
In [53]: df.loc[0, "Full_Name"] = "values"
```

```
In [54]: df
```

```
Out[54]:
```

|   | Full_Name | Age | City     | Salary |
|---|-----------|-----|----------|--------|
| 0 | values    | 28  | New York | 65000  |
| 1 | Anna      | 34  | Paris    | 70000  |
| 2 | Peter     | 29  | Berlin   | 62000  |
| 3 | Linda     | 42  | London   | 85000  |

```
In [55]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Full_Name    4 non-null      object
1   Age          4 non-null      int64
2   City         4 non-null      object
3   Salary       4 non-null      int64
dtypes: int64(2), object(2)
memory usage: 260.0+ bytes
```

```
In [56]: df.describe()
```

```
Out[56]:
```

|       | Age       | Salary       |
|-------|-----------|--------------|
| count | 4.000000  | 4.000000     |
| mean  | 33.250000 | 70500.000000 |
| std   | 6.396614  | 10214.368964 |
| min   | 28.000000 | 62000.000000 |
| 25%   | 28.750000 | 64250.000000 |
| 50%   | 31.500000 | 67500.000000 |
| 75%   | 36.000000 | 73750.000000 |
| max   | 42.000000 | 85000.000000 |

```
In [57]: df.shape
```

```
Out[57]: (4, 4)
```

```
In [58]: import numpy as np
data = {
    'A': [1, 2, np.nan, 4, 5],
    'B': [1, 2, 3, 4, 5],
    'C': [1, 2, 3, np.nan, np.nan],
    'D': [1, np.nan, np.nan, np.nan, 5]
}
```

```
In [59]: df2= pd.DataFrame(data)
```

```
In [60]: df2
```

```
Out[60]:
```

|   | A   | B | C   | D   |
|---|-----|---|-----|-----|
| 0 | 1.0 | 1 | 1.0 | 1.0 |
| 1 | 2.0 | 2 | 2.0 | NaN |
| 2 | NaN | 3 | 3.0 | NaN |
| 3 | 4.0 | 4 | NaN | NaN |
| 4 | 5.0 | 5 | NaN | 5.0 |

```
In [62]: df2.isna().sum()
```

```
Out[62]: A    1  
B     0  
C     2  
D     3  
dtype: int64
```

```
In [65]: df2.dropna() #inplace=True
```

```
Out[65]:
```

|   | A   | B | C   | D   |
|---|-----|---|-----|-----|
| 0 | 1.0 | 1 | 1.0 | 1.0 |

```
In [64]: df2
```

```
Out[64]:
```

|   | A   | B | C   | D   |
|---|-----|---|-----|-----|
| 0 | 1.0 | 1 | 1.0 | 1.0 |
| 1 | 2.0 | 2 | 2.0 | NaN |
| 2 | NaN | 3 | 3.0 | NaN |
| 3 | 4.0 | 4 | NaN | NaN |
| 4 | 5.0 | 5 | NaN | 5.0 |

```
In [66]: df2.fillna(1000)
```

```
Out[66]:
```

|          | <b>A</b> | <b>B</b> | <b>C</b> | <b>D</b> |
|----------|----------|----------|----------|----------|
| <b>0</b> | 1.0      | 1        | 1.0      | 1.0      |
| <b>1</b> | 2.0      | 2        | 2.0      | 1000.0   |
| <b>2</b> | 1000.0   | 3        | 3.0      | 1000.0   |
| <b>3</b> | 4.0      | 4        | 1000.0   | 1000.0   |
| <b>4</b> | 5.0      | 5        | 1000.0   | 5.0      |

```
In [67]: d= {"A":10, "B":20, "C":30, "D":40}
```

```
In [68]: df2.fillna(value=d)
```

```
Out[68]:
```

|          | <b>A</b> | <b>B</b> | <b>C</b> | <b>D</b> |
|----------|----------|----------|----------|----------|
| <b>0</b> | 1.0      | 1        | 1.0      | 1.0      |
| <b>1</b> | 2.0      | 2        | 2.0      | 40.0     |
| <b>2</b> | 10.0     | 3        | 3.0      | 40.0     |
| <b>3</b> | 4.0      | 4        | 30.0     | 40.0     |
| <b>4</b> | 5.0      | 5        | 30.0     | 5.0      |

```
In [71]: m=df2.A.mean()
```

```
In [72]: df2["A"]= df2["A"].fillna(m)
```

```
In [73]: df2
```

```
Out[73]:
```

|          | <b>A</b> | <b>B</b> | <b>C</b> | <b>D</b> |
|----------|----------|----------|----------|----------|
| <b>0</b> | 1.0      | 1        | 1.0      | 1.0      |
| <b>1</b> | 2.0      | 2        | 2.0      | NaN      |
| <b>2</b> | 3.0      | 3        | 3.0      | NaN      |
| <b>3</b> | 4.0      | 4        | NaN      | NaN      |
| <b>4</b> | 5.0      | 5        | NaN      | 5.0      |

```
In [74]: df2.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0    A      5 non-null        float64
1    B      5 non-null        int64
2    C      3 non-null        float64
3    D      2 non-null        float64
dtypes: float64(3), int64(1)
memory usage: 292.0 bytes

```

```

In [75]: data_list = [
        ['John', 28, 'New York', 65000],
        ['Anna', 34, 'Paris', 70000],
        ['Peter', 29, 'Berlin', 62000],
        ['Linda', 42, 'London', 85000]
        ]

```

```

In [76]: pd.DataFrame(data_list)

```

```

Out[76]:
   0   1   2   3
0  John  28  New York  65000
1  Anna  34    Paris  70000
2  Peter  29   Berlin  62000
3  Linda  42   London  85000

```

```

In [77]: d = {'a':10,'b':20,'c':30}

```

```

In [78]: pd.Series(d)

```

```

Out[78]:
a    10
b    20
c    30
dtype: int64

```

```

In [81]: columns = ["Name", "Age", "City", "Salary"]
        data_list = [
        ['John', 28, 'New York', 65000],
        ['Anna', 34, 'Paris', 70000],
        ['Peter', 29, 'Berlin', 62000],
        ['Linda', 42, 'London', 85000]
        ]

```

```

In [82]: pd.DataFrame(data_list, columns=columns)

```



Out[82]:

|   | Name  | Age | City     | Salary |
|---|-------|-----|----------|--------|
| 0 | John  | 28  | New York | 65000  |
| 1 | Anna  | 34  | Paris    | 70000  |
| 2 | Peter | 29  | Berlin   | 62000  |
| 3 | Linda | 42  | London   | 85000  |

|   | Name  | Age | City     | Salary |
|---|-------|-----|----------|--------|
| 0 | John  | 28  | New York | 65000  |
| 1 | Anna  | 34  | Paris    | 70000  |
| 2 | Peter | 29  | Berlin   | 62000  |
| 3 | Linda | 42  | London   | 85000  |

```
In [83]: data = {
    'Date': pd.date_range('2023-01-01', periods=20),
    'Product': ['A', 'B', 'C', 'D'] * 5,
    'Region': ['East', 'West', 'North', 'South', 'East', 'West', 'North', 'South', 'East', 'West', 'North', 'South', 'East', 'West', 'North', 'South', 'East', 'West', 'North', 'South'],
    'Sales': np.random.randint(100, 1000, 20),
    'Units': np.random.randint(10, 100, 20),
    'Rep': ['John', 'Mary', 'Bob', 'Alice', 'John', 'Mary', 'Bob', 'Alice', 'John', 'Mary', 'Bob', 'Alice', 'John', 'Mary', 'Bob', 'Alice', 'John', 'Mary', 'Bob', 'Alice']
}
```

```
In [84]: df3=pd.DataFrame(data)
```

```
In [85]: df3
```

Out[85]:

|           | Date       | Product | Region | Sales | Units | Rep   |
|-----------|------------|---------|--------|-------|-------|-------|
| <b>0</b>  | 2023-01-01 | A       | East   | 360   | 91    | John  |
| <b>1</b>  | 2023-01-02 | B       | West   | 394   | 33    | Mary  |
| <b>2</b>  | 2023-01-03 | C       | North  | 231   | 65    | Bob   |
| <b>3</b>  | 2023-01-04 | D       | South  | 712   | 74    | Alice |
| <b>4</b>  | 2023-01-05 | A       | East   | 439   | 25    | John  |
| <b>5</b>  | 2023-01-06 | B       | West   | 470   | 61    | Mary  |
| <b>6</b>  | 2023-01-07 | C       | North  | 993   | 52    | Bob   |
| <b>7</b>  | 2023-01-08 | D       | South  | 802   | 32    | Alice |
| <b>8</b>  | 2023-01-09 | A       | East   | 672   | 28    | John  |
| <b>9</b>  | 2023-01-10 | B       | West   | 276   | 11    | Mary  |
| <b>10</b> | 2023-01-11 | C       | North  | 865   | 28    | Bob   |
| <b>11</b> | 2023-01-12 | D       | South  | 185   | 54    | Alice |
| <b>12</b> | 2023-01-13 | A       | East   | 902   | 13    | John  |
| <b>13</b> | 2023-01-14 | B       | West   | 904   | 39    | Mary  |
| <b>14</b> | 2023-01-15 | C       | North  | 540   | 24    | Bob   |
| <b>15</b> | 2023-01-16 | D       | South  | 882   | 71    | Alice |
| <b>16</b> | 2023-01-17 | A       | East   | 310   | 22    | John  |
| <b>17</b> | 2023-01-18 | B       | West   | 589   | 54    | Mary  |
| <b>18</b> | 2023-01-19 | C       | North  | 713   | 90    | Bob   |
| <b>19</b> | 2023-01-20 | D       | South  | 991   | 94    | Alice |

In [87]: `df3.head(10)`

Out[87]:

|   | Date       | Product | Region | Sales | Units | Rep   |
|---|------------|---------|--------|-------|-------|-------|
| 0 | 2023-01-01 | A       | East   | 360   | 91    | John  |
| 1 | 2023-01-02 | B       | West   | 394   | 33    | Mary  |
| 2 | 2023-01-03 | C       | North  | 231   | 65    | Bob   |
| 3 | 2023-01-04 | D       | South  | 712   | 74    | Alice |
| 4 | 2023-01-05 | A       | East   | 439   | 25    | John  |
| 5 | 2023-01-06 | B       | West   | 470   | 61    | Mary  |
| 6 | 2023-01-07 | C       | North  | 993   | 52    | Bob   |
| 7 | 2023-01-08 | D       | South  | 802   | 32    | Alice |
| 8 | 2023-01-09 | A       | East   | 672   | 28    | John  |
| 9 | 2023-01-10 | B       | West   | 276   | 11    | Mary  |

In [89]: df3.tail(8)

Out[89]:

|    | Date       | Product | Region | Sales | Units | Rep   |
|----|------------|---------|--------|-------|-------|-------|
| 12 | 2023-01-13 | A       | East   | 902   | 13    | John  |
| 13 | 2023-01-14 | B       | West   | 904   | 39    | Mary  |
| 14 | 2023-01-15 | C       | North  | 540   | 24    | Bob   |
| 15 | 2023-01-16 | D       | South  | 882   | 71    | Alice |
| 16 | 2023-01-17 | A       | East   | 310   | 22    | John  |
| 17 | 2023-01-18 | B       | West   | 589   | 54    | Mary  |
| 18 | 2023-01-19 | C       | North  | 713   | 90    | Bob   |
| 19 | 2023-01-20 | D       | South  | 991   | 94    | Alice |

In [120... df3.sample(5)

Out[120...]

|    | Date       | Product | Region | Sales | Units | Rep   |
|----|------------|---------|--------|-------|-------|-------|
| 11 | 2023-01-12 | D       | South  | 185   | 54    | Alice |
| 10 | 2023-01-11 | C       | North  | 865   | 28    | Bob   |
| 19 | 2023-01-20 | D       | South  | 991   | 94    | Alice |
| 14 | 2023-01-15 | C       | North  | 540   | 24    | Bob   |
| 1  | 2023-01-02 | B       | West   | 394   | 33    | Mary  |

In [ ]: